

Data Types in Java

Understanding type specifications, primitive data structures, and memory bounds

In the previous tutorial, you learned about variables and literals. Now let's understand something critical: **What kind of data can a variable store?** That's where **data types** come in.

1. What Are Data Types?

A **data type** specifies the type of data that a variable can hold.

For example:

```
int dailyPracticeMinutes = 90;
```

Explanation:

- `dailyPracticeMinutes` → Variable name
- `int` → Data type

This explicitly means that the variable can store *only integer values*.

Java is Statically Typed

Java requires every variable to be declared with a data type before it is used.

Valid Declaration VALID

```
double mockAccuracy = 84.5;
```

Invalid Declaration INVALID

```
mockAccuracy = 84.5; // ❌ Error
```

Java must know the data type in advance to perform allocation and type checking.

2. Primitive Data Types in Java

Java has **8 predefined primitive data types**. These are the most basic data storage types built directly into the language.

1) boolean

- **Stores:** Logical values (`true` or `false`)
- **Size:** 1-bit (logical representation)
- **Default value:** `false`
- **Used in:** Conditions, validation checks, and access control logic

```
boolean hasActiveSubscription = true;  
System.out.println(hasActiveSubscription);
```

2) byte

- **Size:** 8-bit signed integer

- **Range:** -128 to 127
- **Default value:** 0
- **Used when:** Memory efficiency matters in large arrays or raw binary streaming

```
byte moduleLevel = 5;
System.out.println(moduleLevel);
```

3) short

- **Size:** 16-bit signed integer
- **Range:** -32,768 to 32,767
- **Default value:** 0
- **Used when:** The integer range is small but larger than `byte`

```
short contestScore = 1500;
System.out.println(contestScore);
```

4) int

- **Size:** 32-bit signed integer
- **Range:** -2^{31} to $2^{31}-1$ (-2,147,483,648 to 2,147,483,647)
- **Default value:** 0
- **Used when:** Standard integral value tracking

```
int totalUsers = 250000;
System.out.println(totalUsers);
```

For most placement problems and general programming, `int` is the default integer choice.

5) long

- **Size:** 64-bit signed integer
- **Range:** -2^{63} to $2^{63}-1$
- **Default value:** 0L
- **Used when:** Storing very large numbers that exceed `int` limits

```
long platformVisits = 150000000000L;
System.out.println(platformVisits);
```

Notice the `L` suffix at the end. Without `L`, Java treats integer literals as `int` and throws a compilation error for oversized values.

6) double

- **Size:** 64-bit IEEE 754 floating-point
- **Default value:** 0.0
- **Used for:** General decimal fractional numbers

```
double averageCompletionTime = 3.75;
System.out.println(averageCompletionTime);
```

⚠ Financial Precision Warning

Never use `double` (or `float`) for precise currency calculations due to binary floating-point rounding errors. Use `BigDecimal` instead.

7) float

- **Size:** 32-bit IEEE 754 floating-point
- **Default value:** `0.0f`
- **Used when:** Less precision is needed or to save memory in massive floating-point arrays

```
float discountRate = 12.5F;  
System.out.println(discountRate);
```

Notice the `F` suffix. Without `F`, Java treats decimal values as `double` by default.

8) char

- **Size:** 16-bit Unicode character
- **Default value:** `' '`
- **Used for:** Storing a single character or Unicode symbol

```
char gradeLevel = 'A';  
System.out.println(gradeLevel);  
  
// Assigning using Unicode escape sequence:  
char symbol = '\u2606'; // Star symbol ★  
System.out.println(symbol);
```

3. Primitive vs Reference Types

The 8 types detailed above are **primitive types**. Java also supports **reference types**, such as:

- `String`
- `Arrays`
- `Classes`
- `Objects`

String (Reference Type)

Strings are not primitive types. They are objects of the `java.lang.String` class.

```
String trackName = "DSA Mastery Track";  
System.out.println(trackName);
```

💡 Why Data Types Matter (Placement Perspective)

In technical interviews and coding assessments, top companies evaluate:

- **Proper data type selection:** Picking memory-efficient and range-appropriate types.
- **Overflow handling:** Avoiding integer wrap-around bug scenarios.
- **Precision issues:** Understanding floating-point representation limitations.
- **Memory optimization:** Choosing appropriate primitives in resource-constrained problems.
- **Type mismatch errors:** Avoiding implicit conversion bugs.

Example: If problem constraints allow values up to 10^{12} , using `int` will cause silent overflow (since `int` max is $\sim 2 \times 10^9$). You must use `long`. Strong data type understanding prevents runtime errors, wrong answers, and edge case failures.

4. Quick Comparison Table

Data Type	Size	Default Value	Example Use Case
boolean	1 bit (logical)	<code>false</code>	Access control, flag conditions
byte	8-bit	<code>0</code>	Small counters, byte stream buffers
short	16-bit	<code>0</code>	Medium range integers
int	32-bit	<code>0</code>	General integer counters, indices
long	64-bit	<code>0L</code>	Large values (e.g., timestamps, massive sums)
float	32-bit	<code>0.0f</code>	Simple floating-point values
double	64-bit	<code>0.0</code>	Precise decimal values
char	16-bit	<code>' '</code>	Single characters, symbols

📝 Practical Exercise

Try creating variables for:

- Number of contests attended
- Average problem-solving time
- Whether premium member or not
- Grade in mock test

Run them inside a Quick Compiler and observe **type behavior**, **literal assignment**, and **output differences**. Consistent hands-on practice builds real clarity.

What's Next?

Now that you understand data types, the next logical steps are: **Type Casting**, **Operators**, and **Taking Input using Scanner**.